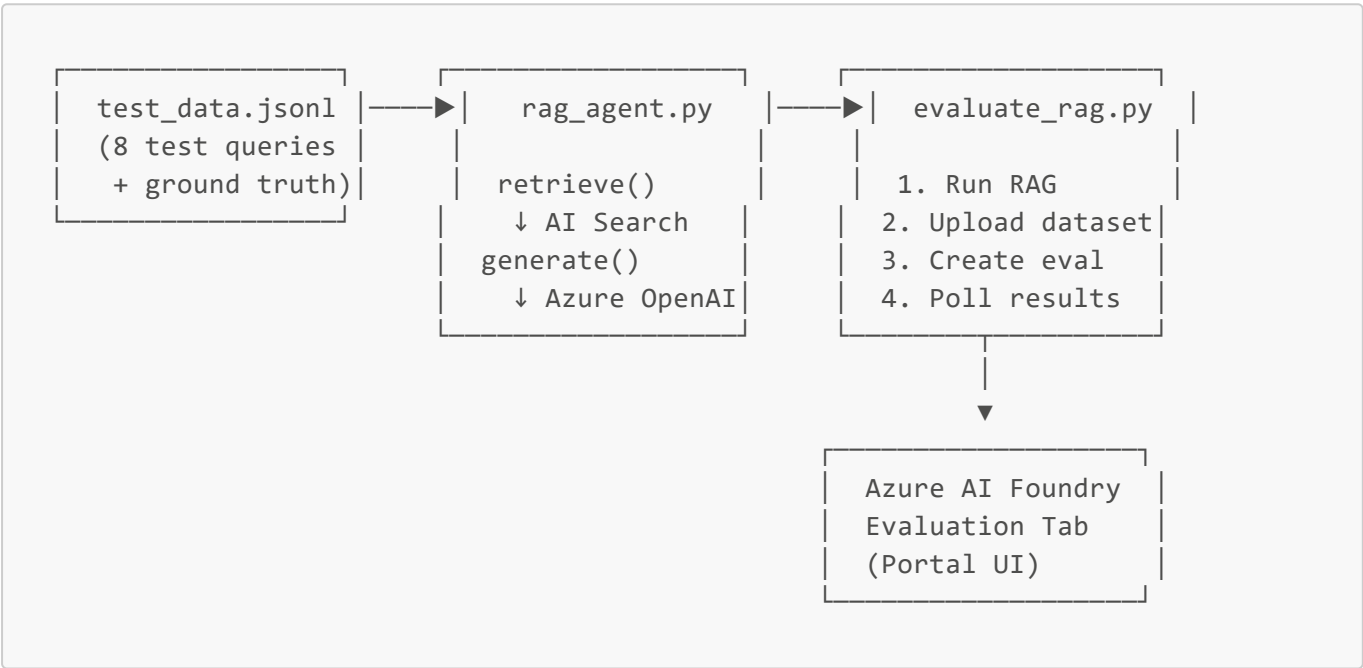


RAG Evaluation with Azure AI Foundry — Comprehensive Guide

Overview

This project demonstrates how to build a RAG (Retrieval-Augmented Generation) pipeline and evaluate its quality using **Azure AI Foundry's cloud evaluation API**. Evaluation results appear directly in the **Foundry portal's Evaluation tab**.

Architecture



File Structure

```
evalagent/
├── setup_search.py      # One-time: create AI Search index + upload docs
├── rag_agent.py         # RAG pipeline: retrieve → generate → answer
├── evaluate_rag.py      # Cloud evaluation via OpenAI Evals API
├── test_data.jsonl     # 8 test queries with ground truth answers
├── requirements.txt     # Python dependencies
├── .env.example        # Environment variable template
├── .env                # Your actual config (git-ignored)
└── .gitignore
```

Prerequisites

Resource	Purpose
Azure AI Search	Stores and retrieves knowledge base documents

Resource	Purpose
Azure OpenAI	GPT-4o deployment for RAG generation + evaluation
Azure AI Foundry Project	New-style Foundry project (CognitiveServices-based, not Hub/ML workspace)
Python 3.10+	Runtime
Azure CLI	Authentication via <code>az login</code>

Important: This uses the **new Foundry project** format (endpoint like `https://<account>.services.ai.azure.com/api/projects/<project>`), NOT the classic ML workspace format.

Setup

1. Create Azure Resources

```
# Create a resource group
az group create --name rg-eval-demo --location eastus2

# Create Azure AI Search
az search service create \
  --name search-eval-demo \
  --resource-group rg-eval-demo \
  --sku basic \
  --location eastus

# Create Azure OpenAI
az cognitiveservices account create \
  --name aoai-eval-demo \
  --resource-group rg-eval-demo \
  --kind OpenAI \
  --sku S0 \
  --location eastus2

# Deploy GPT-4o model
az cognitiveservices account deployment create \
  --name aoai-eval-demo \
  --resource-group rg-eval-demo \
  --deployment-name gpt-4o \
  --model-name gpt-4o \
  --model-version 2024-11-20 \
  --model-format OpenAI \
  --sku-name GlobalStandard \
  --sku-capacity 10
```

2. Create a New Foundry Project

Create a **new-style** Azure AI Foundry project via the [Foundry portal](#):

1. Go to **ai.azure.com** → **Create project**
2. Choose your subscription and resource group
3. This creates an AI Account + Project (CognitiveServices-based)
4. Note the project endpoint from **Settings** → **Project properties**

The endpoint format is:

```
https://<ai-account>.services.ai.azure.com/api/projects/<project-name>
```

3. Assign RBAC Roles

Your user identity needs these roles:

```
# On the AI Search resource
az role assignment create \
  --assignee <your-user-object-id> \
  --role "Search Index Data Contributor" \
  --scope
/subscriptions/<sub>/resourceGroups/<rg>/providers/Microsoft.Search/searchServices/<search>

# On the Azure OpenAI resource
az role assignment create \
  --assignee <your-user-object-id> \
  --role "Cognitive Services OpenAI User" \
  --scope
/subscriptions/<sub>/resourceGroups/<rg>/providers/Microsoft.CognitiveServices/accounts/<aoai>

# On the AI Foundry project (for dataset upload + evaluation)
az role assignment create \
  --assignee <your-user-object-id> \
  --role "Azure AI Developer" \
  --scope
/subscriptions/<sub>/resourceGroups/<rg>/providers/Microsoft.CognitiveServices/accounts/<ai-account>
```

4. Configure Environment

```
cp .env.example .env
# Edit .env with your actual values
```

.env contents:

```
AZURE_SEARCH_ENDPOINT=https://<your-search>.search.windows.net
AZURE_SEARCH_INDEX=contoso-policies
AZURE_OPENAI_ENDPOINT=https://<your-openai>.openai.azure.com
AZURE_OPENAI_DEPLOYMENT=gpt-4o
AZURE_AI_PROJECT_ENDPOINT=https://<your-account>.services.ai.azure.com/api/projects/<your-project>
```

5. Install Dependencies

```
pip install -r requirements.txt
```

6. Login to Azure

```
az login
```

Running the Demo

Step 1: Populate the Search Index (one-time)

```
python setup_search.py
```

This creates an index called **contoso-policies** and uploads 5 sample policy documents:

Document	Topic
Refund Policy	30-day refunds, digital product exceptions
Shipping Policy	Standard, express, overnight, international
Warranty Information	2-year warranty, claim process
Privacy Policy	Data collection, encryption, deletion
Loyalty Program	Points, redemption, Gold tier

Step 2: Run Evaluation

```
python evaluate_rag.py
```

This does two things:

Phase 1 — RAG Pipeline: Runs each of the 8 test queries through the full retrieve → generate pipeline and saves results to `eval_data_with_responses.jsonl`.

Phase 2 — Cloud Evaluation: Uploads results to Foundry, creates an evaluation with 5 built-in evaluators, and polls for results.

Step 3: View Results in Foundry Portal

1. Go to ai.azure.com
2. Open your project
3. Click the **Evaluation** tab
4. Your evaluation "RAG Groundedness Demo" appears in the list

How It Works

RAG Pipeline (`rag_agent.py`)

```
User Query → retrieve() → AI Search (top 3 docs) → context
                                     ↓
← response ← generate() ← Azure OpenAI + context + query
```

- `retrieve(query)` — Searches the AI Search index and returns the top 3 matching document chunks
- `generate(query, context)` — Sends the context + query to GPT-4o with a system prompt that enforces grounded answers
- `ask(query)` — Combines both steps, returns `{"context": ..., "response": ...}`

Authentication uses **Microsoft Entra ID** (DefaultAzureCredential) — no API keys needed.

Cloud Evaluation (`evaluate_rag.py`)

The evaluation uses the **OpenAI Evals API** via `azure-ai-projects` SDK, which is the API the new Foundry portal reads from.

```
# 1. Create project client
project_client = AIProjectClient(endpoint=endpoint, credential=credential)

# 2. Upload dataset
dataset = project_client.datasets.upload_file(name=..., file_path=...)

# 3. Get OpenAI client (with evals support)
client = project_client.get_openai_client()

# 4. Create evaluation definition with testing criteria
eval_object = client.evals.create(
    name="RAG Groundedness Demo",
    data_source_config=data_source_config,
    testing_criteria=testing_criteria,
)
```

```
# 5. Run evaluation against the dataset
eval_run = client.evals.runs.create(
    eval_id=eval_object.id,
    data_source=CreateEvalJSONLRunDataSourceParam(
        type="jsonl",
        source=SourceFileID(type="file_id", id=dataset.id),
    ),
)

# 6. Poll until completed
run = client.evals.runs.retrieve(run_id=..., eval_id=...)
```

Key insight: The new Foundry portal reads evaluations from the **OpenAI Evals API** (`client.evals`), NOT from the classic `azure-ai-evaluation` SDK's `/evaluations/runs` endpoint. Using the wrong API means results won't appear in the portal.

Evaluators

Five built-in evaluators are configured:

Evaluator	What It Measures	Inputs	Threshold
Groundedness	Is the response supported by the retrieved context?	query, context, response	score ≥ 3/5
Relevance	Does the response address the user's query?	query, context, response	score ≥ 3/5
Coherence	Is the response logically structured and readable?	query, response	score ≥ 3/5
Similarity	Does the response match the ground truth answer?	query, response, ground_truth	score ≥ 3/5
F1 Score	Token-level overlap with ground truth	response, ground_truth	score ≥ 0.5

Data Mapping

Each evaluator maps dataset fields using `{{item.<field>}}` syntax:

```
{
  "type": "azure_ai_evaluator",
  "name": "groundedness",
  "evaluator_name": "builtin.groundedness",
  "initialization_parameters": {
    "deployment_name": "gpt-4o",      # LLM used to judge quality
  },
  "data_mapping": {
    "query": "{{item.query}}",      # from dataset
  }
}
```

```

    "context": "{{item.context}}",      # from dataset
    "response": "{{item.response}}",    # from dataset
  },
}

```

What Each Evaluator Uses

Field	groundedness	relevance	coherence	similarity	f1_score
query	✓	✓	✓	✓	—
context	✓	✓	—	—	—
response	✓	✓	✓	✓	✓
ground_truth	—	—	—	✓	✓

- **Groundedness** uses `context` (retrieved docs), NOT `ground_truth` — it checks if the model's answer is supported by what was actually retrieved
- **Similarity** and **F1 Score** use `ground_truth` — they compare the response against the expected reference answer

Test Data Format

`test_data.jsonl` — one JSON object per line:

```

{"query": "What is Contoso's refund policy?", "ground_truth": "Contoso offers a 30-day full refund..."}
{"query": "How much does express shipping cost?", "ground_truth": "Express shipping costs $12.99..."}

```

To add more test cases, append lines to this file with `query` and `ground_truth` fields.

Troubleshooting

Problem	Solution
<code>CredentialUnavailableError: Failed to invoke the Azure CLI</code>	Run <code>az login</code> again. If it times out, the CLI subprocess is slow — the code uses <code>process_timeout=30</code> to handle this.
<code>403 Forbidden</code> on AI Search	Assign <code>Search Index Data Contributor</code> + <code>Search Index Data Reader</code> roles to your identity.
<code>401 Unauthorized</code> on Azure OpenAI	Assign <code>Cognitive Services OpenAI User</code> role.
Evaluation not visible in Foundry portal	Make sure you're using the new Foundry project (not a classic ML workspace). The code uses the OpenAI Evals API

Problem	Solution
	which publishes to the correct endpoint.
<code>azure-ai-evaluation</code> SDK results don't show in portal	That SDK publishes to <code>/evaluations/runs</code> which the new portal doesn't read. Use the <code>azure-ai-projects</code> + OpenAI Evals API approach in this repo instead.
Rate limiting (429 errors)	Increase your Azure OpenAI deployment capacity or wait and retry.

Dependencies

```
azure-ai-projects>=2.0.0    # Foundry project client + OpenAI evals
azure-identity               # Entra ID (passwordless) authentication
azure-search-documents>=11.4 # AI Search client
openai>=1.30.0              # OpenAI SDK (evals types)
python-dotenv                # .env file loading
```